Skills #0023 Vibe Coding

🔐 DEIXE O CÓDIGO DO SEU PROJETO MAIS SEGURO

Um guia prático para devs que não querem aparecer no próximo vazamento.

Favoritar Curtir

Read-me

🔐 Segurança e Proteção de Dados do Usuário

> *Um guia prático para devs que não querem aparecer no próximo vazamento.*

Índice

1. Políticas do Banco de Dados
2. Proteção das APIs
3. Exposição de API Keys e Informações Sensíveis
4. Repositórios GitHub — Boas Práticas para Projetos Importantes
5. Checklist Rápido
6. Security Auditor — Skill para Claude Code

1.1 Princípio do Menor Privilégio

Cada serviço deve ter acesso apenas ao que precisa. Um serviço de leitura de posts **não** deveria poder deletar usuários.

```
-- ❌ Errado: conexão com superuser
DATABASE_URL=postgresql://postgres:senha@host/db

```

```
-- ✅ Certo: usuário com permissões específicas
DATABASE_URL=postgresql://app_readonly:senha@host/db

-- Criando um role restrito no PostgreSQL
CREATE ROLE app_readonly WITH LOGIN PASSWORD 'senha_segura';
GRANT CONNECT ON DATABASE meu_banco TO app_readonly;
GRANT SELECT ON tabela_posts TO app_readonly;
-- Sem acesso a tabela_usuarios, financeiro, etc.
```

1.2 Row Level Security (RLS) — Supabase / PostgreSQL

Ative RLS em **todas** as tabelas que guardam dados de usuários. Isso garante que o usuário A nunca veja os dados do usuário B, mesmo que sua query esteja errada.

```
-- Ativar RLS na tabela
ALTER TABLE documentos ENABLE ROW LEVEL SECURITY;

-- Política: cada usuário só vê seus próprios documentos
CREATE POLICY "usuarios_veem_apenas_seus_docs"
ON documentos
FOR ALL
USING (auth.uid() = user_id);
```

> 💡 ***Supabase**: ative RLS em toda tabela nova pelo dashboard. Se você está usando a chave `service_role` no cliente, **você está bypassando o RLS** — isso é para uso exclusivo no servidor.*

1.3 Nunca Exponha a Chave `service_role` no Frontend

Chave	Onde usar	O que faz
`anon`	Frontend / cliente	Respeitando as políticas RLS
`service_role`	**Só no servidor**	Bypassa RLS — acesso total

1.4 Sanitize Inputs — Previna SQL Injection

```
// ❌ NUNCA faça isso
const resultado = await db.query(
  `SELECT * FROM usuarios WHERE nome = '${nomeDoUsuario}'`
);

// ✅ Use queries parametrizadas
const resultado = await db.query(
  `SELECT * FROM usuarios WHERE nome = $1`,
```

```
  [nomeDoUsuario]
);

// ✅ Com Supabase (já parametrizado automaticamente)
const { data } = await supabase
  .from('usuarios')
  .select('*')
  .eq('nome', nomeDoUsuario);
```

1.5 Backups e Criptografia

- Ative backups automáticos (Supabase oferece isso por padrão nos planos pagos).
- Dados sensíveis (CPF, documentos, dados bancários) devem ser criptografados em repouso.
- Use `pgcrypto` para criptografar campos sensíveis no PostgreSQL.

```
-- Instalando extensão
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Armazenando CPF criptografado
INSERT INTO usuarios (cpf_hash)
VALUES (crypt('123.456.789-00', gen_salt('bf')));
```

---

2. Proteção das APIs

2.1 Autenticação em Todos os Endpoints

Nenhum endpoint que acessa dados de usuários deve ser público sem autenticação.

```
// ❌ Endpoint sem proteção
app.get('/api/usuarios', async (req, res) => {
  const usuarios = await db.query('SELECT * FROM usuarios');
  return res.json(usuarios);
});

// ✅ Middleware de autenticação
import { requireAuth } from '@/middleware/auth';

app.get('/api/usuarios', requireAuth, async (req, res) => {
  const { user } = req; // vem do middleware
  const dados = await db.query(
    'SELECT * FROM dados WHERE user_id = $1',
```

```
    [user.id]
  );
  return res.json(dados);
});
```

2.2 Rate Limiting

Proteja seus endpoints contra abuso, brute force e scraping.

```
// Usando express-rate-limit
import rateLimit from 'express-rate-limit';

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos
  max: 100, // máximo 100 requests por IP
  message: 'Muitas requisições. Tente novamente em 15 minutos.',
});

// Aplicar em toda a API
app.use('/api/', limiter);

// Rate limit mais restrito para login
const loginLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1 hora
  max: 10, // 10 tentativas
});
app.use('/api/auth/login', loginLimiter);
```

2.3 Validação de Input com Zod

Valide **todos** os dados que chegam na sua API antes de processar.

```
import { z } from 'zod';

const criarUsuarioSchema = z.object({
  nome: z.string().min(2).max(100),
  email: z.string().email(),
  senha: z.string().min(8).regex(/[A-Z]/).regex(/[0-9]/),
  cpf: z.string().regex(/^\d{3}\.\d{3}\.\d{3}-\d{2}$/),
});

app.post('/api/usuarios', async (req, res) => {
  const validacao = criarUsuarioSchema.safeParse(req.body);

  if (!validacao.success) {
    return res.status(400).json({ erro: validacao.error.flatten() });
```

```
  }

  // Agora sim, dados limpos e validados
  const { nome, email, senha, cpf } = validacao.data;
  // ...
});
```

2.4 CORS Configurado Corretamente

```
import cors from 'cors';

// ❌ CORS aberto para tudo (perigoso em produção)
app.use(cors());

// ✅ Apenas origens confiáveis
app.use(cors({
  origin: [
    'https://meusite.com.br',
    'https://www.meusite.com.br',
    // Em dev:
    process.env.NODE_ENV === 'development' ? 'http://localhost:3000' : null,
  ].filter(Boolean),
  credentials: true,
}));
```

2.5 Headers de Segurança

Use `helmet` para setar headers de segurança automaticamente:

```
import helmet from 'helmet';

app.use(helmet());
// Isso adiciona: X-Frame-Options, X-XSS-Protection,
// Strict-Transport-Security, Content-Security-Policy, etc.
```

---

3. Exposição de API Keys e Informações Sensíveis

3.1 Regra de Ouro: Nunca no Código-Fonte

```
# ❌ NUNCA isso no seu código
const OPENAI_KEY = "sk-proj-xxxxxxxxxxxxxxxxxxx";
const SUPABASE_SECRET = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...";

# ✅ Sempre via variáveis de ambiente
const OPENAI_KEY = process.env.OPENAI_API_KEY;
const SUPABASE_SECRET = process.env.SUPABASE_SERVICE_ROLE_KEY;
```

3.2 Arquivo `.env` — Nunca Commitar

```
# .gitignore — adicione ANTES do primeiro commit
.env
.env.local
.env.production
.env.*.local

# Arquivos de configuração com segredos
config/secrets.json
*.pem
*.key
```

Crie sempre um `.env.example` para documentar quais variáveis são necessárias:

```
# .env.example (este SIM vai para o git)
DATABASE_URL=postgresql://user:password@host:5432/dbname
SUPABASE_URL=https://seu-projeto.supabase.co
SUPABASE_ANON_KEY=sua-chave-anon-aqui
OPENAI_API_KEY=sua-chave-openai-aqui
JWT_SECRET=um-segredo-longo-e-aleatorio
```

3.3 Variáveis Públicas vs. Privadas (Next.js)

```
# ✅ Prefixo NEXT_PUBLIC_ = vai para o browser (use só para coisas realmente
NEXT_PUBLIC_SUPABASE_URL=https://projeto.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ... # anon key é OK aqui com RLS ativado

# ✅ Sem prefixo = só no servidor
SUPABASE_SERVICE_ROLE_KEY=eyJ... # NUNCA no cliente
```

```
STRIPE_SECRET_KEY=sk_live_...    # NUNCA no cliente
DATABASE_URL=postgresql://...    # NUNCA no cliente
```

3.4 Rotação de Chaves Comprometidas

Se uma chave vazou, **revogue imediatamente**:

1. Acesse o dashboard do serviço (OpenAI, Supabase, Stripe...)
2. Revogue a chave comprometida
3. Gere uma nova chave
4. Atualize em todos os ambientes (dev, staging, produção)
5. Verifique nos logs se houve uso indevido

> ⚠️ ***Se você commitou uma secret key no git**: deletar o arquivo não resolve — ela continua no histórico. Use `git filter-repo` ou o BFG Repo Cleaner para limpar o histórico.*

3.5 Gerenciamento de Segredos em Produção

Plataforma	Como guardar segredos
Vercel	Dashboard → Settings → Environment Variables
Railway	Dashboard → Variables
Render	Dashboard → Environment
AWS	AWS Secrets Manager ou Parameter Store
VPS	Arquivo `.env` fora do diretório público, permissão 600

```
# Em VPS, proteja o arquivo .env
chmod 600 /var/www/app/.env
chown www-data:www-data /var/www/app/.env
```

3.6 Auditoria com `git-secrets` ou `truffleHog`

```
# Instalar git-secrets para prevenir commits com segredos
brew install git-secrets  # macOS
git secrets --install
git secrets --register-aws  # detecta chaves AWS

```

```
# Escanear histórico do repositório
trufflehog git https://github.com/seu-usuario/seu-repo
```

---

4. Repositórios GitHub — Boas Práticas para Projetos Importantes

4.1 Repositório Privado por Padrão

Projetos com clientes, dados reais ou lógica de negócio sensível **sempre privados**.

```
GitHub → Repositório → Settings → Danger Zone → Change visibility → Private
```

4.2 Branch Protection Rules

Proteja a branch `main` contra pushes diretos e force-push.

```
GitHub → Repositório → Settings → Branches → Add rule

✅ Branch name pattern: main
✅ Require a pull request before merging
✅ Require approvals: 1 (ou mais)
✅ Dismiss stale pull request approvals when new commits are pushed
✅ Require status checks to pass before merging
✅ Require branches to be up to date before merging
✅ Do not allow bypassing the above settings
✅ Restrict who can push to matching branches
```

4.3 Gerenciar Acessos com Cuidado

```
GitHub → Repositório → Settings → Collaborators and teams

Boas práticas:
- Dê acesso mínimo necessário (Read vs. Write vs. Admin)
- Remova ex-colaboradores imediatamente
- Use teams em vez de adicionar pessoas diretamente
- Revise acessos periodicamente
```

4.4 GitHub Secrets para CI/CD

Nunca coloque secrets nos arquivos de workflow. Use GitHub Secrets:

```yaml
# .github/workflows/deploy.yml

# ❌ Errado
- name: Deploy
  run: |
    SUPABASE_KEY=eyJabc123... npm run deploy

# ✅ Certo — usando GitHub Secrets
- name: Deploy
  env:
    SUPABASE_SERVICE_ROLE_KEY: ${{ secrets.SUPABASE_SERVICE_ROLE_KEY }}
    DATABASE_URL: ${{ secrets.DATABASE_URL }}
  run: npm run deploy
```

```
Configurar em: GitHub → Repositório → Settings → Secrets and variables → Action
```

4.5 Ativar Dependabot e Security Alerts

```
GitHub → Repositório → Settings → Security

✅ Dependency graph
✅ Dependabot alerts
✅ Dependabot security updates
✅ Secret scanning (automático em repos públicos, disponível em private com Gi
```

4.6 Code Scanning com CodeQL

```yaml
# .github/workflows/codeql.yml
name: CodeQL Analysis

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  schedule:
    - cron: '30 1 * * 0'  # Toda domingo

jobs:
  analyze:
```

```
    name: Analyze
    runs-on: ubuntu-latest
    permissions:
      security-events: write

    steps:
      - uses: actions/checkout@v4

      - name: Initialize CodeQL
        uses: github/codeql-action/init@v3
        with:
          languages: javascript, typescript

      - name: Perform CodeQL Analysis
        uses: github/codeql-action/analyze@v3
```

4.7 Revisão de Pull Requests

```
# .github/CODEOWNERS
# Define quem deve revisar mudanças em arquivos críticos

# Arquivos de configuração e infraestrutura
/.env.example @seu-usuario
/docker-compose*.yml @seu-usuario
/.github/ @seu-usuario

# Arquivos de migração do banco
/supabase/migrations/ @seu-usuario @dev-senior

# Código de autenticação e pagamentos
/src/auth/ @seu-usuario
/src/payments/ @seu-usuario
```

---

5. Checklist Rápido

Use antes de fazer deploy ou abrir um PR:

Banco de Dados

- ☐ RLS ativado em todas as tabelas com dados de usuário
- ☐ Usuário de banco com permissões mínimas necessárias
- ☐ Chave `service_role` apenas no servidor, nunca no cliente
- ☐ Queries parametrizadas (sem concatenação de strings SQL)

- ☐ Backups automáticos configurados

APIs

- ☐ Todos os endpoints autenticados (exceto rotas públicas intencionais)
- ☐ Rate limiting configurado
- ☐ Validação de input em todos os endpoints
- ☐ CORS configurado com origens específicas
- ☐ Headers de segurança com helmet

Secrets e Variáveis de Ambiente

- ☐ `.env` no `.gitignore`
- ☐ Nenhuma chave hardcoded no código
- ☐ `.env.example` atualizado
- ☐ Variáveis `NEXT_PUBLIC_` apenas para dados realmente públicos
- ☐ Segredos em produção via dashboard da plataforma (Vercel, Railway, etc.)

GitHub

- ☐ Repositório privado (se necessário)
- ☐ Branch protection na `main`
- ☐ GitHub Secrets para CI/CD
- ☐ Dependabot ativado
- ☐ CODEOWNERS configurado para arquivos críticos
- ☐ Acesso de colaboradores revisado e atualizado

> ***Lembre-se:*** *Segurança não é uma feature que você adiciona no final. É uma prática que você mantém do primeiro commit ao deploy em produção.*

---

6. Security Auditor — Skill para Claude Code

A skill `security-auditor` transforma o Claude Code em um auditor de segurança que varre seu projeto automaticamente, identifica vulnerabilidades e entrega um relatório com severidade, trecho real do código problemático e correção sugerida.

6.1 Pré-requisitos

- **Claude Code** instalado (docs.anthropic.com/claude-code)
- Arquivo `security-auditor.skill` baixado

Para instalar o Claude Code caso ainda não tenha:

```
npm install -g @anthropic-ai/claude-code
```

Confirme que está funcionando:

```
claude --version
```

6.2 Instalando a Skill

Opção A — Via comando direto (recomendado)

```
claude skill install security-auditor.skill
```

Opção B — Copiando manualmente para a pasta de skills

```
# macOS / Linux
mkdir -p ~/.claude/skills
cp security-auditor.skill ~/.claude/skills/

# Windows (PowerShell)
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\.claude\skills"
Copy-Item security-auditor.skill "$env:USERPROFILE\.claude\skills\"
```

Verificar se a skill foi instalada:

```
claude skill list
# Deve aparecer: security-auditor
```

6.3 Usando a Skill no Projeto

Navegue até a raiz do projeto que deseja auditar e abra o Claude Code:

```
cd /caminho/do/seu/projeto
claude
```

Dentro do Claude Code, use linguagem natural — a skill dispara automaticamente:

```
# Auditoria completa
> audita a segurança do projeto

# Verificações específicas
> verifica se tem alguma API key exposta no código
> checa se o banco está com RLS ativado
> faz um security review antes do deploy
> tem alguma falha de autenticação nos endpoints?
> verifica as dependências vulneráveis

# Investigação pontual
> esse arquivo aqui tem alguma vulnerabilidade? [cola o código]
> como está o CORS configurado no projeto?
```

6.4 Entendendo o Relatório Gerado

A skill gera um relatório com 4 níveis de severidade:

Ícone	Nível	O que significa	Ação
🔴	**Crítico**	Exposição imediata de dados ou acesso indevido	Corrigir antes de qualquer deploy
🟠	**Alto**	Falha exploitável com algum esforço	Corrigir na próxima sprint
🟡	**Médio**	Má prática que pode virar problema	Planejar correção
🟢	**Baixo**	Melhoria de segurança recomendada	Backlog

Exemplo de output:

```
## 🔴 CRÍTICO — API Key Hardcoded

**Arquivo:** src/lib/openai.ts — Linha 3
**Problema:** Chave da OpenAI hardcoded no código-fonte

Código problemático:
  const openai = new OpenAI({ apiKey: "sk-proj-****" });

Correção:
  const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

E no .env:
  OPENAI_API_KEY=sua-chave-aqui
```

> ⚠️ *Por segurança, a skill **mascara automaticamente** os valores reais de secrets encontrados no output (`sk-****` , `eyJ****`). Ela identifica a localização, não expõe o valor.*

6.5 Módulos de Verificação

A skill cobre 7 áreas automaticamente:

```
✅ Módulo 1 — Secrets e API Keys hardcoded
✅ Módulo 2 — Banco de dados (SQL Injection, RLS, service_role no cliente)
✅ Módulo 3 — Autenticação e endpoints desprotegidos
✅ Módulo 4 — CORS, headers de segurança, helmet
✅ Módulo 5 — Dependências com vulnerabilidades conhecidas (npm audit)
✅ Módulo 6 — Exposição de dados no frontend (NEXT_PUBLIC_, console.log)
✅ Módulo 7 — Configuração do repositório GitHub (.gitignore, workflows)
```

6.6 Fluxo Recomendado — Antes de Todo Deploy

```
# 1. Abrir o Claude Code na raiz do projeto
cd meu-projeto && claude

# 2. Pedir a auditoria completa
> faz um security review completo antes do deploy

# 3. Corrigir os achados críticos e altos

# 4. Rodar novamente para confirmar
> verifica novamente se os problemas críticos foram resolvidos

# 5. Deploy com mais tranquilidade 🚀
```

6.7 Dicas de Uso

Combine com o checklist do tutorial: Após a auditoria automática, use o Checklist Rápido desta documentação para validação manual dos pontos que a ferramenta não consegue checar automaticamente (como lógica de autorização complexa).

Projetos grandes: Para projetos com muitos arquivos, seja mais específico:

```
> audita apenas a pasta src/api em busca de endpoints desprotegidos
> verifica só os arquivos de migração do Supabase
> checa as variáveis de ambiente do projeto
```

Integrar no fluxo de PR: Peça a auditoria ao revisar Pull Requests que mexem em autenticação, banco ou configuração:

```
> esse PR mexe em autenticação — verifica se introduziu alguma vulnerabilidade
```

Arquivo disponível
Clique para baixar

Baixar

Comentários (3)

Escreva um comentário...

Comentar

Usuário 3d atrás
Brabo! Me salvou 🙏
Responder

Usuário 2d atrás
Bom demais irmão, vim aqui pelo reels sobre o cara lá que desdenhou dos devs.
Responder

Usuário 1d atrás
top!!
Responder

← Voltar para a biblioteca